Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Εξάμηνο 8ο
Ψηφιακά Συστήματα VLSI

2η Εργαστηριακή Άσκηση

Σχεδίαση Μονάδων Υλικού με την Τεχνική Pipelining

Μέλη Ομάδας

Καβαδάκης Σωτήρης 03122035
Κοντοπίδης Δημήτρης 03122025

Αθήνα, Μάρτιος 2026

Ερώτημα 1: Σύγχρονος Πλήρης Αθροιστής (Synchronous Full Adder)

Περιγραφή Κυκλώματος

Ο Σύγχρονος Πλήρης Αθροιστής (Synchronous Full Adder) είναι ένα ακολουθιακό κύκλωμα που εκτελεί πρόσθεση τριών bits (δύο bits εισόδου και ένα κρατούμενο εισόδου) με τις εξόδους να καταχωρούνται σε flip-flops στη θετική ακμή του ρολογιού. Σε αντίθεση με τον συνδυαστικό Full Adder, οι έξοδοι ενημερώνονται μόνο στη θετική ακμή του σήματος ρολογιού, εισάγοντας καθυστέρηση ενός κύκλου.

Υλοποιήθηκε με περιγραφή συμπεριφοράς (Behavioral) χρησιμοποιώντας process με ευαισθησία στο ρολόι.

Είσοδοι:

- i_clk: Σήμα ρολογιού
- i_a: Πρώτο bit εισόδου
- i_b: Δεύτερο bit εισόδου
- i_cin: Κρατούμενο εισόδου

Έξοδοι:

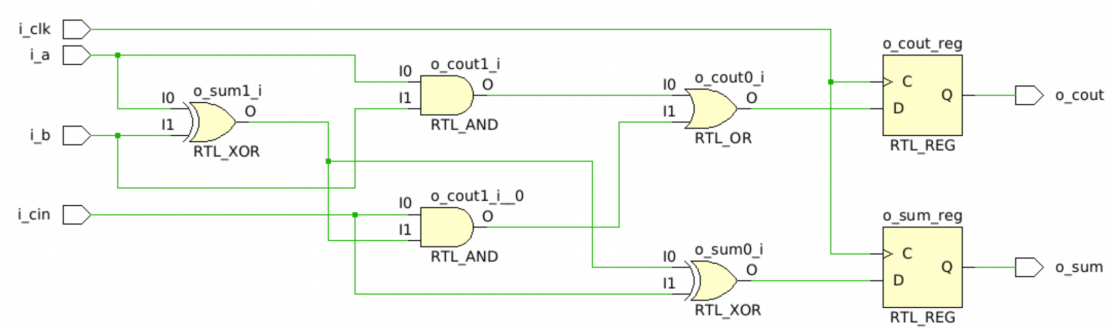
- o_sum: Άθροισμα ($S = A \oplus B \oplus C_{in}$)
- o_cout: Κρατούμενο εξόδου ($C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$)

Ο πίνακας αλήθειας του Synchronous Full Adder:

i_a	i_b	i_cin	o_sum	o_cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Πίνακας 1Πίνακας αλήθειας Synchronous Full Adder

a) Σχεδιάγραμμα Λειτουργίας (RTL Schematic)



Σχήμα RTL σχηματικό του Synchronous Full Adder

Το RTL σχηματικό αποτελείται από:

- Μία πύλη **XOR** για τον υπολογισμό του ενδιάμεσου $A \oplus B$
- Δύο πύλες **AND** για τους όρους $A \cdot B$ και $C_{in} \cdot (A \oplus B)$
- Μία πύλη **OR** για τον υπολογισμό του κρατούμενου C_{out}
- Μία πύλη **XOR** για τον υπολογισμό του τελικού αθροίσματος $(A \oplus B) \oplus C_{in}$
- Δύο **RTL_REG** (flip-flops) που καταχωρούν τα o_sum και o_cout στη θετική ακμή του ρολογιού

b) Κώδικας VHDL - Behavioral Model

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sync_full_adder is
    Port (
        i_clk  : in STD_LOGIC;
        i_a    : in STD_LOGIC;
        i_b    : in STD_LOGIC;
        i_cin  : in STD_LOGIC;
        o_sum  : out STD_LOGIC;
        o_cout : out STD_LOGIC
    );
end sync_full_adder;

architecture Behavioral of sync_full_adder is
begin
```

```
    process(i_clk)
    begin
        if rising_edge(i_clk) then
            o_sum <= (i_a XOR i_b XOR i_cin);
            o_cout <= (i_a AND i_b) OR (i_cin AND (i_a XOR i_b));
        end if;
    end process;
end Behavioral;
```

Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sync_full_adder_tb is
end entity;

architecture tb of sync_full_adder_tb is

    component sync_full_adder is
        Port (
            i_clk       : in  STD_LOGIC;
            i_a         : in  STD_LOGIC;
            i_b         : in  STD_LOGIC;
            i_cin       : in  STD_LOGIC;
            o_sum       : out STD_LOGIC;
            o_cout      : out STD_LOGIC
        );
    end component;

    signal i_Clk         : std_logic := '0';
    signal i_A           : std_logic := '0';
    signal i_B           : std_logic := '0';
    signal i_Carry_in    : std_logic := '0';
    signal o_Sum         : std_logic;
    signal o_Carry_out   : std_logic;

    constant CLK_PERIOD : time := 10 ns;

begin

    DUT: sync_full_adder
        port map (
            i_clk   => i_Clk,
            i_a     => i_A,
            i_b     => i_B,
```

```
            i_cin   => i_Carry_in ,
            o_sum   => o_Sum ,
            o_cout  => o_Carry_out
        );

    CLK_PROCESS: process
    begin
        i_Clk <= '0';
        wait for CLK_PERIOD / 2;
        i_Clk <= '1';
        wait for CLK_PERIOD / 2;
    end process;

    STIMULUS: process
    begin
        i_A <= '0'; i_B <= '0'; i_Carry_in <= '0';
        wait for CLK_PERIOD;
        assert (o_Sum = '0' and o_Carry_out = '0')
            report "FAIL: 0+0+0 expected Sum=0, Carry=0";

        i_A <= '0'; i_B <= '0'; i_Carry_in <= '1';
        wait for CLK_PERIOD;
        assert (o_Sum = '1' and o_Carry_out = '0')
            report "FAIL: 0+0+1 expected Sum=1, Carry=0";

        i_A <= '0'; i_B <= '1'; i_Carry_in <= '0';
        wait for CLK_PERIOD;
        assert (o_Sum = '1' and o_Carry_out = '0')
            report "FAIL: 0+1+0 expected Sum=1, Carry=0";

        i_A <= '0'; i_B <= '1'; i_Carry_in <= '1';
        wait for CLK_PERIOD;
        assert (o_Sum = '0' and o_Carry_out = '1')
            report "FAIL: 0+1+1 expected Sum=0, Carry=1";

        i_A <= '1'; i_B <= '0'; i_Carry_in <= '0';
        wait for CLK_PERIOD;
        assert (o_Sum = '1' and o_Carry_out = '0')
            report "FAIL: 1+0+0 expected Sum=1, Carry=0";

        i_A <= '1'; i_B <= '0'; i_Carry_in <= '1';
        wait for CLK_PERIOD;
        assert (o_Sum = '0' and o_Carry_out = '1')
            report "FAIL: 1+0+1 expected Sum=0, Carry=1";

        i_A <= '1'; i_B <= '1'; i_Carry_in <= '0';
        wait for CLK_PERIOD;
```

```
        assert (o_Sum = '0' and o_Carry_out = '1')
            report "FAIL: 1+1+0 expected Sum=0, Carry=1";

        i_A <= '1'; i_B <= '1'; i_Carry_in <= '1';
        wait for CLK_PERIOD;
        assert (o_Sum = '1' and o_Carry_out = '1')
            report "FAIL: 1+1+1 expected Sum=1, Carry=1";

        report "All tests passed!";
        wait;
    end process;

end architecture tb;
```

Ανάλυση Αποτελεσμάτων Testbench

Το testbench ελέγχει εξαντλητικά όλους τους $2^3 = 8$ δυνατούς συνδυασμούς εισόδου. Για κάθε συνδυασμό, οι είσοδοι εφαρμόζονται και μετά από έναν κύκλο ρολογιού (10 ns) ελέγχονται οι καταχωρημένες έξοδοι μέσω εντολών assert. Η καθυστέρηση ενός κύκλου αντικατοπτρίζει τη σύγχρονη φύση του κυκλώματος. Εφόσον δεν υπήρξε κάποιο assert error κατά την εκτέλεση, επιβεβαιώθηκε η ορθή λειτουργία.

c) Κρίσιμο Μονοπάτι (Critical Path)

Unconstrained Paths - NONE - clk - Setup

Name	Slack ^1	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock
Path 1	∞	2	3	2	i_a	o_sum_reg/D	1.932	1.132	0.800	∞	input port clock
Path 2	∞	2	3	2	i_cin	o_cout_reg/D	1.906	1.106	0.800	∞	input port clock

Σχήμα Κρίσιμο μονοπάτι του Synchronous Full Adder

Το κρίσιμο μονοπάτι του Synchronous Full Adder είναι από την είσοδο i_a προς τον καταχωρητή o_sum_reg/D με συνολική καθυστέρηση **1.932 ns** (Logic Delay: 1.132 ns, Net Delay: 0.800 ns). Το μονοπάτι διέρχεται από 2 επίπεδα λογικής (δύο πύλες XOR) και 3 routes. Πρόκειται για μονοπάτι τύπου input-to-register, καθώς οι είσοδοι δεν είναι καταχωρημένες.

Ερώτημα 2: Σύγχρονος Αθροιστής 4 bits με Pipeline

Περιγραφή Κυκλώματος

Ο Pipelined Αθροιστής 4 bits εκτελεί πρόσθεση δύο 4-bit αριθμών με κρατούμενο εισόδου, χρησιμοποιώντας την τεχνική pipeline. Η υλοποίηση βασίζεται σε τέσσερα instances του Synchronous Full Adder (Ερώτημα 1), συνδεδεμένα σε αλυσίδα ripple carry, με επιπλέον pipeline registers για την ευθυγράμμιση των δεδομένων.

Το κύκλωμα μπορεί να δέχεται ένα νέο ζεύγος εισόδων σε κάθε κύκλο ρολογιού και να παράγει ορθό αποτέλεσμα σε κάθε κύκλο μετά από αρχική καθυστέρηση $T_{latency} = 4$ κύκλους.

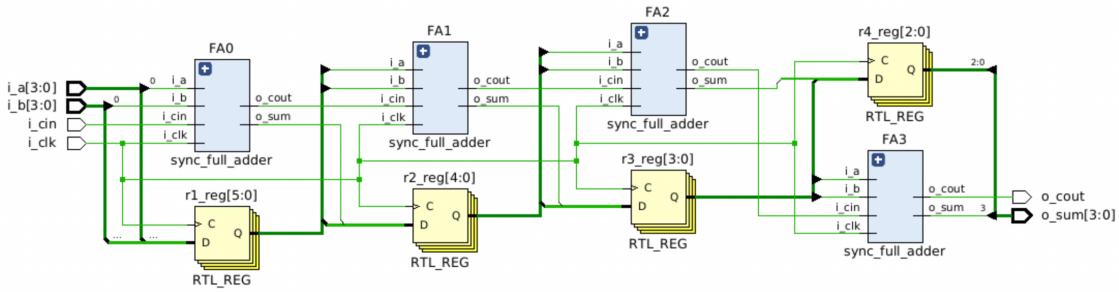
Είσοδοι:

- i_clk: Σήμα ρολογιού
- i_a(3:0): Πρώτος 4-bit αριθμός
- i_b(3:0): Δεύτερος 4-bit αριθμός
- i_cin: Κρατούμενο εισόδου

Έξοδοι:

- o_sum(3:0): 4-bit άθροισμα
- o_cout: Κρατούμενο εξόδου

a) Σχεδιάγραμμα Λειτουργίας (RTL Schematic)



Σχήμα RTL σχηματικό του Pipelined Adder 4 bits

Η αρχιτεκτονική pipeline χρησιμοποιεί τέσσερα στάδια, ένα για κάθε bit πρόσθεσης. Μεταξύ των σταδίων προστέθηκαν pipeline registers (r1, r2, r3, r4) για τους εξής λόγους:

- **r1 (6 bits)**: Καταχωρεί τα bits εισόδου (b3, a3, b2, a2, b1, a1) που δεν έχουν ακόμη επεξεργαστεί, ώστε να είναι διαθέσιμα στο σωστό στάδιο pipeline.
- **r2 (5 bits)**: Καταχωρεί τα bits (b3, a3, b2, a2) καθώς και το αποτέλεσμα s0 από το FA0, ευθυγραμμίζοντάς τα χρονικά με το FA1.
- **r3 (4 bits)**: Καταχωρεί τα bits (b3, a3) και τα αποτελέσματα (s1, s0) για ευθυγράμμιση με το FA2.
- **r4 (3 bits)**: Καταχωρεί τα αποτελέσματα (s2, s1, s0) για ευθυγράμμιση με το FA3.

Οι pipeline registers είναι απαραίτητοι γιατί κάθε Full Adder χρειάζεται ένα κύκλο ρολογιού για να υπολογίσει το αποτέλεσμά του. Χωρίς τους ενδιάμεσους καταχωρητές, τα bits εισόδου των ανώτερων βαθμίδων θα είχαν ήδη αλλάξει τιμή πριν χρησιμοποιηθούν, καθώς νέα δεδομένα εισέρχονται σε κάθε κύκλο.

b) Κώδικας VHDL

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity pipelined_adder is
    port (
        i_clk  : in  std_logic;
        i_a    : in  std_logic_vector(3 downto 0);
        i_b    : in  std_logic_vector(3 downto 0);
        i_cin  : in  std_logic;
        o_sum  : out std_logic_vector(3 downto 0);
        o_cout : out std_logic
    );
end pipelined_adder;

architecture Behavioral of pipelined_adder is

    component sync_full_adder is
        port (
            i_clk  : in  std_logic;
            i_a    : in  std_logic;
            i_b    : in  std_logic;
            i_cin  : in  std_logic;
            o_sum  : out std_logic;
            o_cout : out std_logic
        );
    end component;

    -- Carry signals between FA stages
```

```
    signal c1, c2, c3 : std_logic;

    -- Sum outputs from each FA
    signal s_int : std_logic_vector(3 downto 0);

    -- Pipeline registers
    signal r1 : std_logic_vector(5 downto 0) := (others => '0');
    signal r2 : std_logic_vector(4 downto 0) := (others => '0');
    signal r3 : std_logic_vector(3 downto 0) := (others => '0');
    signal r4 : std_logic_vector(2 downto 0) := (others => '0');

begin

    -- Pipeline register process
    process(i_clk)
    begin
        if rising_edge(i_clk) then
            r1 <= i_b(3) & i_a(3) & i_b(2) & i_a(2) & i_b(1) & i_a(1);
            r2 <= r1(5) & r1(4) & r1(3) & r1(2) & s_int(0);
            r3 <= r2(4) & r2(3) & s_int(1) & r2(0);
            r4 <= s_int(2) & r3(1) & r3(0);
        end if;
    end process;

    FA0: sync_full_adder
        port map (
            i_clk => i_clk, i_a => i_a(0), i_b => i_b(0),
            i_cin => i_cin, o_sum => s_int(0), o_cout => c1
        );

    FA1: sync_full_adder
        port map (
            i_clk => i_clk, i_a => r1(0), i_b => r1(1),
            i_cin => c1, o_sum => s_int(1), o_cout => c2
        );

    FA2: sync_full_adder
        port map (
            i_clk => i_clk, i_a => r2(1), i_b => r2(2),
            i_cin => c2, o_sum => s_int(2), o_cout => c3
        );

    FA3: sync_full_adder
        port map (
            i_clk => i_clk, i_a => r3(2), i_b => r3(3),
            i_cin => c3, o_sum => s_int(3), o_cout => o_cout
        );
```

```
    -- Align all sums
    o_sum(0) <= r4(0);
    o_sum(1) <= r4(1);
    o_sum(2) <= r4(2);
    o_sum(3) <= s_int(3);

end Behavioral;
```

Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity pipelined_adder_tb is
end entity;

architecture tb of pipelined_adder_tb is

    component pipelined_adder is
        port (
            i_clk  : in  std_logic;
            i_a    : in  std_logic_vector(3 downto 0);
            i_b    : in  std_logic_vector(3 downto 0);
            i_cin  : in  std_logic;
            o_sum  : out std_logic_vector(3 downto 0);
            o_cout : out std_logic
        );
    end component;

    signal i_clk  : std_logic := '0';
    signal i_a    : std_logic_vector(3 downto 0) := (others => '0');
    signal i_b    : std_logic_vector(3 downto 0) := (others => '0');
    signal i_cin  : std_logic := '0';
    signal o_sum  : std_logic_vector(3 downto 0);
    signal o_cout : std_logic;

    constant CLK_PERIOD : time := 10 ns;

begin

    DUT: pipelined_adder
        port map (
            i_clk => i_clk, i_a => i_a, i_b => i_b,
            i_cin => i_cin, o_sum => o_sum, o_cout => o_cout
```

```
        );

    CLK_PROCESS: process
    begin
        i_clk <= '0';
        wait for CLK_PERIOD / 2;
        i_clk <= '1';
        wait for CLK_PERIOD / 2;
    end process;

    STIMULUS: process
        variable expected_cout : std_logic;
    begin
        for a in 0 to 15 loop
            for b in 0 to 15 loop
                for cin in 0 to 1 loop
                    i_a <= std_logic_vector(to_unsigned(a, 4));
                    i_b <= std_logic_vector(to_unsigned(b, 4));
                    if cin = 1 then
                        i_cin <= '1';
                    else
                        i_cin <= '0';
                    end if;
                    wait for 4 * CLK_PERIOD;
                    if (a + b + cin) >= 16 then
                        expected_cout := '1';
                    else
                        expected_cout := '0';
                    end if;
                    assert (o_sum = std_logic_vector(
                            to_unsigned((a + b + cin) mod 16, 4)))
                        and (o_cout = expected_cout)
                        report "FAIL: " & integer'image(a) & "+"
                            & integer'image(b) & "+"
                            & integer'image(cin);
                end loop;
            end loop;
        end loop;
        report "All 512 tests passed!";
        wait;
    end process;

end architecture tb;
```

Ανάλυση Αποτελεσμάτων Testbench

Το testbench ελέγχει εξαντλητικά όλους τους $16 \times 16 \times 2 = 512$ δυνατούς συνδυασμούς εισόδου. Για κάθε συνδυασμό, εφαρμόζονται οι είσοδοι και μετά από 4 κύκλους ρολογιού (η καθυστέρηση pipeline $T_{latency}$), ελέγχεται ότι το αποτέλεσμα είναι ορθό μέσω assert. Εφόσον δεν υπήρξε κάποιο assert error κατά την εκτέλεση, επιβεβαιώθηκε η ορθή λειτουργία του κυκλώματος για όλες τις δυνατές εισόδους.

c) Κρίσιμο Μονοπάτι και Χρονική Ανάλυση (Critical Path & Timing)

Intra-Clock Paths - clk - Setup

Name	Slack ^1	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	7.940	1	2	2	FA0/o_cout_reg/C	r3_reg[1]_srl2/D	1.836	0.751	1.085
Path 2	8.288	0	1	1	r3_reg[0]_srl3/CLK	r4_reg[0]/D	1.617	1.617	0.000
Path 3	8.288	0	1	1	r3_reg[1]_srl2/CLK	r4_reg[1]/D	1.617	1.617	0.000
Path 4	8.346	1	2	2	r1_reg[1]/C	FA1/o_cout_reg/D	1.503	0.751	0.752
Path 5	8.346	1	2	2	r2_reg[2]/C	FA2/o_cout_reg/D	1.503	0.751	0.752
Path 6	8.346	1	2	2	FA1/o_cout_reg/C	FA2/o_sum_reg/D	1.503	0.751	0.752
Path 7	8.346	1	2	2	r3_reg[3]/C	FA3/o_cout_reg/D	1.503	0.751	0.752
Path 8	8.346	1	2	2	FA2/o_cout_reg/C	FA3/o_sum_reg/D	1.503	0.751	0.752
Path 9	8.795	0	1	1	r1_reg[2]/C	r2_reg[1]/D	0.790	0.456	0.334
Path 10	8.795	0	1	1	r1_reg[3]/C	r2_reg[2]/D	0.790	0.456	0.334

Σχήμα Κρίσιμο μονοπάτι του Pipelined Adder

Το κρίσιμο μονοπάτι του Pipelined Adder είναι από τον καταχωρητή FA0/o_cout_reg/C προς τον καταχωρητή r3_reg[1]_srl2/D με συνολική καθυστέρηση **2.060 ns** (clock period 10 ns, WNS = 7.940 ns). Το μονοπάτι διέρχεται από 1 επίπεδο λογικής και 2 routes. Πρόκειται για μονοπάτι register-to-register εντός ενός pipeline stage — ακριβώς αυτό που επιτυγχάνει η τεχνική pipeline.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	7.940 ns	Worst Hold Slack (WHS):	0.139 ns	Worst Pulse Width Slack (WPWS):	4.020 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	15	Total Number of Endpoints:	15	Total Number of Endpoints:	24

All user specified timing constraints are met.

Σχήμα Design Timing Summary του Pipelined Adder

Στο Design Timing Summary (Σχήμα) παρατηρούμε ότι όλα τα timing constraints ικανοποιούνται:

- **Setup**: WNS = 7.940 ns, δηλαδή $T_{clk} - T_{critical} = 10.0 - 2.060 = 7.940$ ns slack. Αυτό σημαίνει ότι το κύκλωμα θα μπορούσε να λειτουργήσει σε συχνότητα έως $f_{max} = 1/2.060$ ns $\approx$ 485 MHz.

- **Hold**: WHS = 0.139 ns > 0, άρα δεν υπάρχουν hold violations.
- **Pulse Width**: WPWS = 4.020 ns > 0, εντός προδιαγραφών.
- Σύνολο 15 setup endpoints και 24 pulse width endpoints, 0 failing — πλήρης επιτυχία.

Κατανάλωση Πόρων (Resource Utilization)

Name	Slice LUTs (17600)	Slice Registers (35200)	Bonded IOB (100)	BUFGCTRL (32)
pipelined_adder	10	21	15	1
FA0 (sync_full_adder)	2	1	0	0
FA1 (sync_full_adder_0)	2	1	0	0
FA2 (sync_full_adder_1)	2	2	0	0
FA3 (sync_full_adder_2)	1	2	0	0

Σχήμα Resource Utilization του Pipelined Adder

Από τον πίνακα utilization (Σχήμα) προκύπτει ότι ο Pipelined Adder χρησιμοποιεί συνολικά **10 Slice LUTs**, **21 Slice Registers** (flip-flops), **15 Bonded IOBs** και **1 BUFGCTRL**. Η κατανομή ανά instance είναι:

- Κάθε sync_full_adder (FA0--FA3) χρησιμοποιεί 2 LUTs (ένα για το sum, ένα για το carry) και 1-2 flip-flops (για τις εξόδους o_sum και o_cout).
- Οι FA2 και FA3 έχουν 2 registers αντί για 1, καθώς τα carry bits τους περνούν μέσω pipeline registers στα επόμενα stages.
- Τα υπόλοιπα flip-flops ($21 - 6 = 15$) αντιστοιχούν στους pipeline registers r1--r4 που ευθυγραμμίζουν τα ενδιάμεσα αποτελέσματα μεταξύ stages.

Σύγκριση με Παράλληλο Αθροιστή (Lab 1)

Θεωρώντας τον Παράλληλο Αθροιστή 4 bits της Εργαστηριακής Άσκησης 1 ως σύγχρονο κύκλωμα, η χρονική καθυστέρηση του κρίσιμου μονοπατιού του (5.970 ns) αντιστοιχεί στην ελάχιστη περίοδο ρολογιού που θα μπορούσε να υποστηρίξει.

	Parallel Adder (Lab 1)	**Pipelined Adder (Lab 2)**
Critical Path	5.970 ns	2.060 ns
Μέγιστη συχνότητα (f_{max})	∼167 MHz	∼485 MHz
Latency	1 κύκλος (5.970 ns)	4 κύκλοι (8.240 ns)
Throughput	1 αποτέλεσμα / 5.970 ns	1 αποτέλεσμα / 2.060 ns
Slice LUTs	∼4	10
Slice Registers (FFs)	0	21
Bonded IOBs	14	15

Πίνακας 2Σύγκριση Parallel Adder vs Pipelined Adder

Αιτιολόγηση Διαφορών

- **Χρονική καθυστέρηση (Critical Path)**: Ο Pipelined Adder έχει σημαντικά μικρότερο critical path (2.060 ns vs 5.970 ns) διότι η τεχνική pipeline σπάει το μεγάλο συνδυαστικό μονοπάτι (ripple carry μέσω 4 FA) σε μικρότερα τμήματα. Κάθε pipeline stage περιέχει μόνο ένα επίπεδο λογικής (1 full adder), με αποτέλεσμα η καθυστέρηση να περιορίζεται στην καθυστέρηση ενός μόνο full adder συν τον χρόνο routing. Αυτό επιτρέπει ∼2.9x υψηλότερη συχνότητα ρολογιού (485 MHz vs 167 MHz).

- **Throughput vs Latency**: Ο Pipelined Adder έχει μεγαλύτερο latency σε απόλυτο χρόνο ($4 \times 2.060 = 8.240$ ns vs 5.970 ns), αφού το αποτέλεσμα χρειάζεται 4 κύκλους για να εξέλθει από το pipeline. Ωστόσο, μετά τη γέμιση του pipeline, παράγεται ένα νέο αποτέλεσμα κάθε 2.060 ns αντί κάθε 5.970 ns. Αυτό αντιστοιχεί σε ∼2.9x μεγαλύτερο throughput, κρίσιμο πλεονέκτημα σε εφαρμογές συνεχούς ροής δεδομένων (streaming).

- **Κατανάλωση πόρων (Area)**: Ο Pipelined Adder χρησιμοποιεί 2.5x περισσότερα LUTs (10 vs ∼4) και 21 flip-flops (vs 0). Τα LUTs αυξάνονται επειδή κάθε sync_full_adder χρειάζεται τη δική του συνδυαστική λογική, ενώ στον παράλληλο αθροιστή ο synthesizer μπορεί να βελτιστοποιήσει την κοινή λογική. Τα 21 flip-flops προέρχονται από:

  - 8 FFs στους 4 sync_full_adders (2 ανά FA: o_sum και o_cout)
  - 13 FFs στους pipeline registers r1(6 bits), r2(5 bits), r3(4 bits), r4(3 bits) — αν και η σύνθεση βελτιστοποιεί κάποια με SRL (Shift Register LUT), γι' αυτό βλέπουμε λιγότερα register resources σε ορισμένα instances.

  Αυτό αποτελεί το κλασικό **trade-off area vs speed**: η αύξηση του throughput επιτυγχάνεται με κόστος σε κατανάλωση πόρων FPGA.

- **Bonded IOBs**: Ο Pipelined Adder χρησιμοποιεί 15 IOBs (1 επιπλέον σε σχέση με τον Parallel Adder), λόγω της εισόδου ρολογιού i_clk που απαιτεί dedicated I/O pin μέσω BUFGCTRL.